

06 · GraphQL: consultas anidadas, variables y OAuth

Sumamos cuatro cosas sobre c1:

1. **Variables** tipadas (`$page: Int!`) en lugar de interpolar strings.
2. **Campos anidados**: cada orden con su `cliente` y sus `items` , y de cada ítem su `producto` .
En REST esto serían varias llamadas; aquí es **una sola** por página.
3. Autenticación con **OAuth Bearer** (reutilizamos el flujo de b2) en vez de API key.
4. Las **reseñas de entrega** (`comentarios(tipo: "post_compra")`) consultadas por orden.

```
import csv
import os

import requests

BASE = "http://localhost:3000"
GQL = f"{BASE}/api/graphql"
TIMEOUT = 30

# --- OAuth: obtener un access token (igual que en b2) ---
tok = requests.post(
    f"{BASE}/api/oauth/token",
    data={
        "grant_type": "client_credentials",
        "client_id": "scraper-demo",
        "client_secret": "scraper-demo-secret",
    },
    timeout=TIMEOUT,
).json()["access_token"]

HEADERS = {"Authorization": f"Bearer {tok}"}
print("Token OAuth:", tok[:28], "...")

def gql(query, variables=None):
    r = requests.post(
        GQL, json={"query": query, "variables": variables or {}}, headers=HEADERS,
        timeout=TIMEOUT
    )
    r.raise_for_status()
    cuerpo = r.json()
    if "errors" in cuerpo:
        raise RuntimeError(cuerpo["errors"])
    return cuerpo["data"]
```

Token OAuth: eyJhbGciOiJIUzI1NiJ9.eyJzY29 ...

Una query anidada con variables

ordenes -> items (lista) -> cada item con cliente e items { producto }. Traemos las 3 primeras para inspeccionar la forma de la respuesta.

```
QUERY_ORDENES = """
query PaginaOrdenes($page: Int!, $pageSize: Int!) {
  ordenes(page: $page, pageSize: $pageSize) {
    items {
      id numero fecha subTotal igv total
      cliente { id nombre apellidos ciudad pais }
      items {
        productoId cantidad subTotal
        producto { codigo nombre categoria }
      }
    }
  }
  pageInfo { page totalPages total }
}
"""

primera = gql(QUERY_ORDENES, {"page": 1, "pageSize": 3})["ordenes"]
print("pageInfo:", primera["pageInfo"])

o = primera["items"][0]
print(f"\nOrden {o['numero']} ({o['fecha']})")
print(f"  Cliente: {o['cliente']['nombre']} {o['cliente']['apellidos']} - {o['cliente']['ciudad']}, {o['cliente']['pais']}")
print(f"  Total: S/ {o['total']}")
for it in o["items"]:
    print(f"    {it['cantidad']:>2} x {it['producto']['nombre'][:40]:40s} S/ {it['subTotal']}")
```

pageInfo: {'page': 1, 'totalPages': 100, 'total': 300}

Orden OR-00300 (2026-08-10)

Cliente: Mateo Sebastian Dominguez Silva - Ciudad de Mexico, Mexico

Total: S/ 70311.48

4 x DJI Air 3	S/ 27996
1 x Samsung T7 SSD Portatil 1TB	S/ 399
2 x Dell XPS 13 9340 Intel Core i7	S/ 12998
3 x LG OLED55C3 55 pulgadas	S/ 14997
4 x SteelSeries Arctis Nova 7	S/ 3196

Recorrer todas las páginas de órdenes

```
def todas_las_ordenes(page_size=50):
    items = []
    page = 1
    while True:
        d = gql(QUERY_ORDENES, {"page": page, "pageSize": page_size})["ordenes"]
        items.extend(d["items"])
        info = d["pageInfo"]
        print(f"  página {info['page']:>2}/{info['totalPages']} (total {len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1
```

```
ordenes = todas_las_ordenes()
print(f"\n{len(ordenes)} órdenes")
```

```
página 1/6 (total 50/300)
página 2/6 (total 100/300)
página 3/6 (total 150/300)
página 4/6 (total 200/300)
página 5/6 (total 250/300)
página 6/6 (total 300/300)
```

300 órdenes

Reseñas de entrega por orden

`comentarios(tipo: "post_compra", ordenId: N)` devuelve las reseñas sobre la entrega de esa orden. No todas las órdenes tienen: iteramos todas y nos quedamos con las que sí traen reseñas.

```
QUERY_RESENAS = """
query ResenasEntrega($ordenId: Int!) {
  comentarios(tipo: "post_compra", ordenId: $ordenId) {
    id clienteId productoId calificacion texto fecha
    clienteNombre clienteApellidos
  }
}
"""

resenas = []
for o in ordenes:
    for c in gql(QUERY_RESENAS, {"ordenId": o["id"]})["comentarios"]:
        resenas.append({
            "id": c["id"],
            "orden_id": o["id"],
            "orden_numero": o["numero"],
            "cliente_id": c["clienteId"],
            "cliente_nombre": f"{c['clienteNombre']} {c['clienteApellidos']}",
            "producto_id": c["productoId"],
            "calificacion": c["calificacion"],
            "fecha": c["fecha"],
            "texto": c["texto"],
        })

ordenes_con_resena = len({r["orden_id"] for r in resenas})
print(f"{len(resenas)} reseñas de entrega, repartidas en {ordenes_con_resena} de {len(ordenes)} órdenes")
```

110 reseñas de entrega, repartidas en 74 de 300 órdenes

Guardar en `data/graphql_ordenes.csv` y `data/graphql_resenas_entrega.csv`

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)

ruta_ordenes = os.path.join(DATA_DIR, "graphql_ordenes.csv")
with open(ruta_ordenes, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
```

```

        f,
        fieldnames=["id", "numero", "fecha", "cliente_id", "cliente_nombre",
"cliente_pais", "num_items", "sub_total", "igv", "total"],
    )
    writer.writeheader()
    for o in ordenes:
        cli = o["cliente"]
        writer.writerow({
            "id": o["id"], "numero": o["numero"], "fecha": o["fecha"],
            "cliente_id": cli["id"], "cliente_nombre": f"{cli['nombre']}
{cli['apellidos']}",
            "cliente_pais": cli["pais"], "num_items": len(o["items"]),
            "sub_total": o["subTotal"], "igv": o["igv"], "total": o["total"],
        })

ruta_resenas = os.path.join(DATA_DIR, "graphql_resenas_entrega.csv")
with open(ruta_resenas, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=list(resenas[0].keys()))
    writer.writeheader()
    writer.writerows(resenas)

print("Guardado:", ruta_ordenes)
print("Guardado:", ruta_resenas)

```

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/graphql_ordenes.csv

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/graphql_resenas_entrega.csv

Una sola query anidada reemplazó decenas de llamadas REST. Falta el caso en que **no hay API** y la página exige iniciar sesión: eso es el último notebook, con Playwright.